

Московский авиационный институт (национальный исследовательский университет)

Институт «Компьютерные науки и прикладная математика»

Кафедра «Вычислительная математика и программирование»

Лабораторная работа №2

по курсу «Дискретный анализ»

на тему «Сбалансированные деревья»

Студент: Строков Георгий Владимирович

Группа: М8О-208БВ-24

Преподаватель: Макаров Н.К.

Проверил: Сахарин Н.А.

Дата: 2 мая 2026

Оценка: _____

Подпись: _____

Москва, 2026

1 Постановка задачи

Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64} - 1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Для всех операций, в случае возникновения системной ошибки, программа должна вывести строку, начинающуюся с «ERROR:» и описывающую на английском языке возникшую ошибку.

Различия вариантов заключаются только в используемых структурах данных: PATRICIA.

2 Формат ввода/вывода

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ word 34 — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- word — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

! Save /path/to/file — сохранить словарь в бинарном компактном представлении на диск в файл, указанный параметром команды. В случае успеха, программа должна вывести «OK», в случае неудачи выполнения операции, программа должна вывести описание ошибки.

! Load /path/to/file — загрузить словарь из файла. Предполагается, что файл был ранее подготовлен при помощи команды Save. В случае успеха, программа должна вывести строку «OK», а загруженный словарь должен заменить текущий (с которым происходит работа); в случае неуспеха, должна быть выведена диагностика, а рабочий словарь должен остаться без изменений. Кроме системных ошибок, программа должна корректно обрабатывать случаи несовпадения формата указанного файла и представления данных словаря во внешнем файле.

3 Метод решения (Алгоритм)

3.1 Общая концепция Patricia Trie

Patricia Trie (Practical Algorithm To Retrieve Information Coded in Alphanumeric) – это разновидность бинарного дерева, в котором узлы содержат не целые ключи, а указатели на ветвление по отдельным битам ключа. В отличие от обычного бинарного дерева поиска,

здесь сравнение ключей происходит побитово, а каждый внутренний узел определяет позицию бита (bit index), по которой производится разветвление.

В данной реализации структура узла определяется следующим образом:

Листинг 1: Структура узла

```
1 struct Node {
2     Node *left_, *right_;
3     Key key_;
4     T value_;
5     std::ptrdiff_t bit_;
6     std::ptrdiff_t queue_index;
7 };
```

Корневой узел имеет специальную роль: его поле `bit_` равно -1, как корневого узла, с которого начинается любая итерация по дереву, а `right_` равно `nullptr`.

3.2 Битовый доступ к ключам

Класс `Digitizer` преобразует строку в последовательность битов. Строка рассматривается как массив символов, каждый символ занимает 5 бит (т.е. используется только кодировка A–Z и a–z, что соответствует 32 возможным значениям). Методы класса:

- `bool operator()(const vector<char>& str, ptrdiff_t index)` – возвращает значение индекса-го бита (начиная с 0) от конца строки. Это позволяет одинаково обрабатывать строки разной длины.
- `ptrdiff_t operator()(const vector<char>& first, const vector<char>& second)` – находит позицию первого несовпадающего бита между двумя строками (сравнение регистронезависимое). Если строки совпадают, возвращает -1. Если одна строка является префиксом другой, возвращает позицию бита, где более длинная строка имеет лишний бит.

Таким образом, сравнение ключей сводится к поиску первого отличающегося бита.

3.3 Поиск элемента

Поиск `find()`:

1. Начинаем с узла `root_ -> left_`.
2. Спускаемся по дереву: на каждом шаге смотрим на текущий узел `currunt`. Если его `bit_` меньше или равен предыдущему сохранённому биту, значит алгоритм поиска завершен. Вызывается `digitizer_(key, currunt->key_)`. Если результат -1 (ключи равны), возвращаем указатель на значение, иначе ключ не найден.
3. Иначе продолжаем спуск: выбираем направление в зависимости от значения `digitizer_(key, currunt->bit_)` (0 – левый потомок, 1 – правый).

3.4 Вставка элемента

Вставка `insert()` выполняется следующим образом:

1. Если дерево пусто, создаётся новый узел, который становится корнем, его `left_` указывает на самого себя, `right_` равен `nullptr`.
2. Иначе сначала производится поиск места вставки, аналогично поиску, но с сохранением пути. Находится узел, где обнаружено расхождение битов (или полное совпадение, тогда вставка невозможна – дубликат).
3. После того как определен бит расхождения `diff_bit`, необходимо найти точку в дереве, куда следует вставить новый узел. Для этого ветвление начинается от корня и продвигается вниз, пока не встретим узел, у которого `bit_` больше `diff_bit` или пока не перейдет по обратной ссылке.
4. Новый узел создаётся с ключом и значением, его `bit_` = `diff_bit`. В зависимости от значения бита `diff_bit` у нового ключа, один из его потомков указывает на сам новый узел (замыкание), а другой – на найденный ранее узел `next`.
5. Родительский узел (тот, у которого `bit_` меньше `diff_bit`) перенаправляет свою ссылку на новый узел.

3.5 Удаление элемента

Удаление `erase()` – наиболее сложная операция в Patricia Trie. Общая идея:

1. Найти удаляемый узел `deleteNode`, пройдя по дереву, руководствуясь битами ключ.
2. Найти узел `upToDelete`, который ссылается на удаляемый узел, и его родителя `upParent`.
3. Найти узел `upToUp` – узел, который ссылается на `upToDelete`.
4. Перестроить ссылки: заменить `upToDelete` на `deleteNode` в нужной ветке.
5. Обменять содержимое (ключ и значение) между `deleteNode` и `upToDelete`.
6. Удалить физически узел `upToDelete`.
7. Переопределить ссылки для `upParent` и `upToUp`.

Алгоритм гарантирует сохранение свойств Patricia Trie после удаления.

3.6 Сохранение и загрузка

Для сохранения дерева в бинарный файл используется обход в ширину (BFS). Все узлы нумеруются (индексы в очереди). В файл записывается:

- размер дерева (количество узлов);
- для каждого узла – длина ключа, сам ключ, значение, поле bit_, индексы левого и правого потомка (или -1, если потомок отсутствует).

При загрузке файла последовательно создаются узлы, после чего индексы преобразуются в указатели. Корнем становится узел с индексом 0.

Используется собственный контейнер `mylib::vector`, который предоставляет минимальный набор методов, необходимых для работы (`push_back`, `operator[]`, `size`, `data` и т.д.). Это упрощает сериализацию.

4 Тесты

Входные данные:

```
+ a 1
+ A 2
+ aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa 18446744073709551615
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
A
- A
a
```

Выходные данные:

OK
Exist
OK
OK: 18446744073709551615
OK: 1
OK
NoSuchWord

5 Вывод

Временная сложность операций

- L — длина ключа в битах;
- N — количество ключей в дереве.

Поиск (find). Алгоритм спускается по дереву, на каждом шаге анализируя один бит ключа. Количество итераций не зависит от общего числа узлов, а определяется исключительно длиной ключа. В худшем случае потребуется выполнить L шагов. Таким образом, сложность поиска составляет $O(L)$.

Вставка (insert). Сначала выполняется поиск места для вставки за $O(L)$, затем находится точка расхождения битов и производится перестроение указателей за константное время. Следовательно, вставка также имеет сложность $O(L)$.

Удаление (erase). Алгоритм требует найти удаляемый узел и узел-лист с искомым ключом, выполнив при этом два обхода дерева. Каждый обход занимает $O(L)$ времени. Все операции перестроения указателей и обмена данными выполняются за $O(1)$. Итоговая сложность удаления — $O(L)$.